

OPERATING SYSTEMS LAB

System Call Experiments

<i>Detail</i>	<i>Information</i>
Name	Sayan Ali
Roll NO.	2401010163
Program	B.Tech Computer Science and Engineering
Section	B

Index of Experiments

<i>No.</i>	<i>Experiment</i>	<i>System call(s)</i>
1	Process ID and Parent Process ID	getpid(), getppid()
2	Creating a process	fork()
3	Parent waiting for child	wait()
4	Writing to a file	open(), write(), close()
5	Reading from a file	open(), read(), close()
6	Replacing a process image	exec()
7	Complete process creation	fork(), wait(), getpid()

Experiment 1: getpid() and getppid()

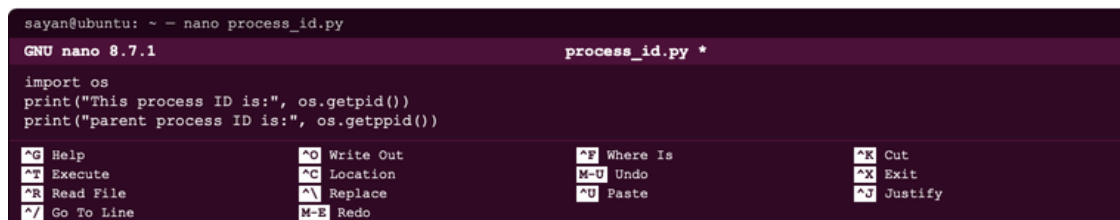
Aim To demonstrate how a process can obtain its own process ID and its parent's process ID.

Procedure

1. Create a Python file: nano process_id.py
2. Enter the program given below.
3. Save the file: press **Ctrl + O**, then **Enter**, then **Ctrl + X**.
4. Run the program: python3 process_id.py

Program

```
import os
print("Process ID:", os.getpid())
print("Parent Process ID:", os.getppid())
```



Output



The process IDs will be different on every computer and on every run.

Explanation

<i>Function</i>	<i>Purpose</i>
os.getpid()	Returns the ID of the current process
os.getppid()	Returns the ID of the parent process

Result: The program printed the process ID of the running Python process and the ID of its parent process (the shell).

Experiment 2: Creating a Process Using fork()

Aim To create a child process using the Linux fork() system call.

Procedure

1. Create the file: nano fork_demo.py
2. Write the program given below.
3. Save with **Ctrl + O, Enter, Ctrl + X**.
4. Run: python3 fork_demo.py

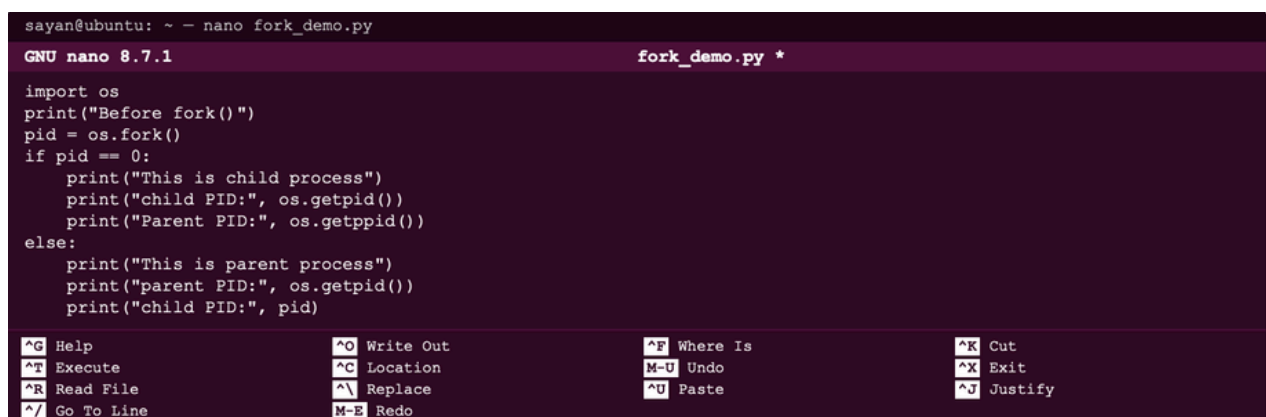
Program

```
import os

print("Before fork()")

pid = os.fork()

if pid == 0:
    print("This is the child process")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())
else:
    print("This is the parent process")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)
```



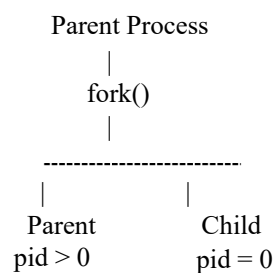
```
sayan@ubuntu: ~ -- nano fork_demo.py
GNU nano 8.7.1                                fork_demo.py *
import os
print("Before fork()")
pid = os.fork()
if pid == 0:
    print("This is child process")
    print("child PID:", os.getpid())
    print("Parent PID:", os.getppid())
else:
    print("This is parent process")
    print("parent PID:", os.getpid())
    print("child PID:", pid)
^G Help      ^O Write Out  ^F Where Is   ^K Cut
^T Execute    ^C Location   M-U Undo      ^X Exit
^R Read File  ^\ Replace    ^U Paste      ^J Justify
^_/ Go To Line M-E Redo
```

Output

```
sayan@ubuntu: ~  
sayan@ubuntu:~$ nano fork_demo.py  
sayan@ubuntu:~$ python3 fork_demo.py  
Before fork()  
This is parent process  
parent PID: 6892  
child PID: 6893  
This is child process  
child PID: 6893  
Parent PID: 6892  
sayan@ubuntu:~$
```

"Before fork()" is printed only once, because it runs before the child exists. After fork(), the parent and the child both continue from the same point, so the parent block and the child block are both printed.

Explanation



The value returned by fork() tells each process which one it is:

- **0**: the executing process is the child
- **Positive value**: the executing process is the parent, and the value is the child's PID
- **Negative value**: process creation failed

Result: A child process was created with fork(), and both the parent and the child printed their own process IDs.

Experiment 3: Parent Waiting for Child Using wait()

Aim To demonstrate process synchronization, where the parent waits for the child process to finish.

Procedure

1. Create the file: nano wait_demo.py
2. Write the program given below and save it.
3. Run: python3 wait_demo.py

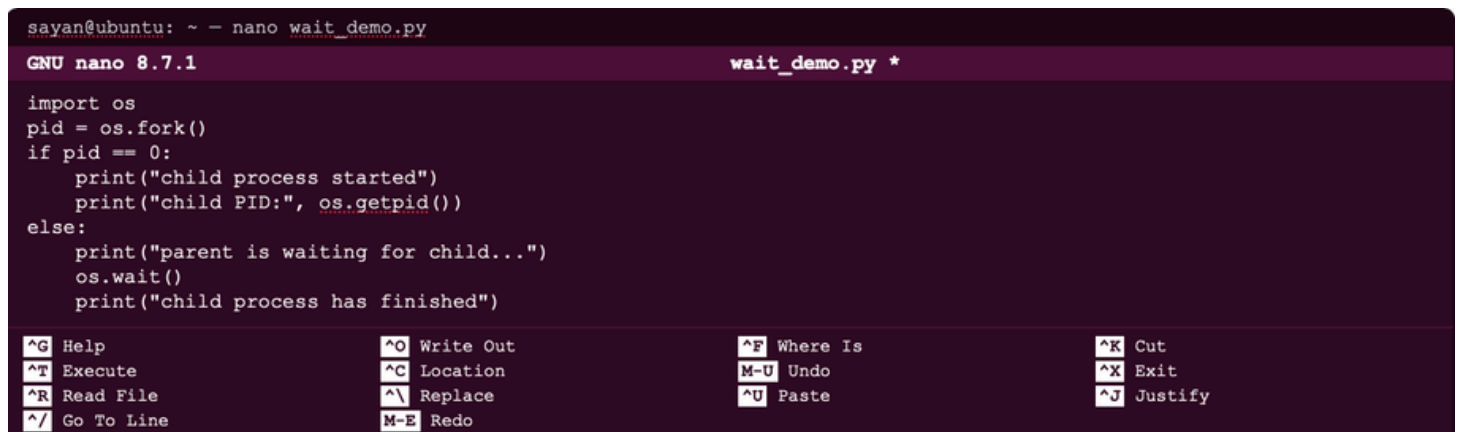
Program

```
import os

pid = os.fork()

if pid == 0:
    print("Child process started")
    print("Child PID:", os.getpid())

else:
    print("Parent is waiting for child...")
    os.wait()
    print("Child process has finished")
```



```
sayan@ubuntu: ~ - nano wait_demo.py
GNU nano 8.7.1                               wait_demo.py *
import os
pid = os.fork()
if pid == 0:
    print("child process started")
    print("child PID:", os.getpid())
else:
    print("parent is waiting for child...")
    os.wait()
    print("child process has finished")

^G Help      ^O Write Out ^F Where Is  ^K Cut
^T Execute   ^C Location  ^M-U Undo    ^X Exit
^R Read File ^\ Replace   ^U Paste     ^J Justify
^/_ Go To Line ^-E Redo
```

Output

```
sayan@ubuntu: ~  
sayan@ubuntu:~$ nano wait_demo.py  
sayan@ubuntu:~$ python3 wait_demo.py  
parent is waiting for child...  
child process started  
child PID: 5318  
child process has finished  
sayan@ubuntu:~$ █
```

Explanation

The parent calls `os.wait()`, which makes it wait until the child terminates. Without this synchronization, the parent may finish before the child does. Here the line "Child process has finished" is always printed last, after the child has completed.

Result: The parent process waited for the child process to terminate using `wait()`, and then continued its execution.

Experiment 4: File System Calls

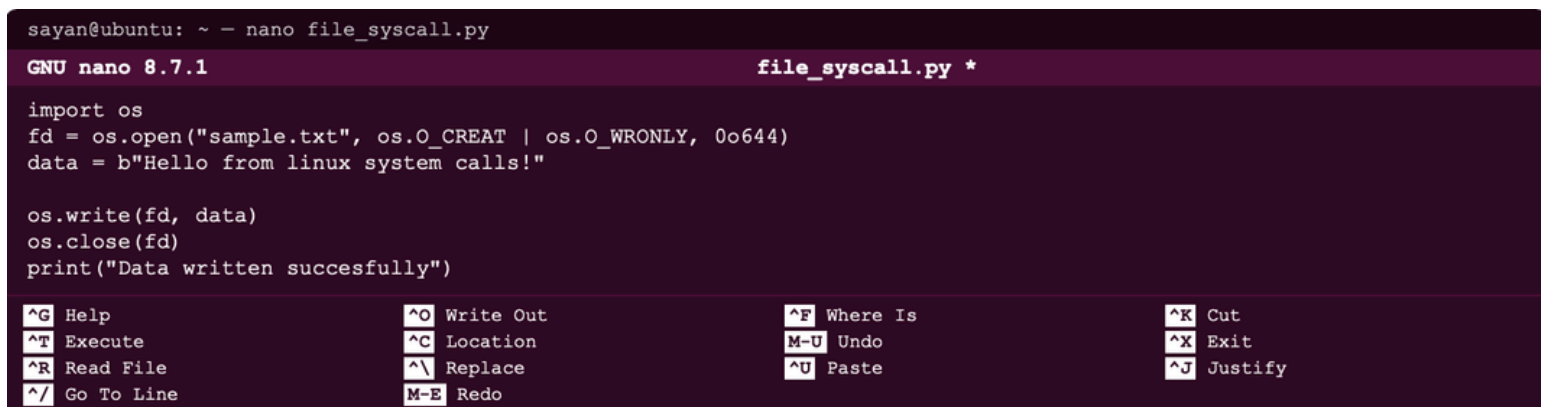
Aim To create a file and write data into it using the Linux system calls `open()`, `write()` and `close()`.

Procedure

1. Create the file: `nano file_syscall.py`
2. Write the program given below and save it.
3. Run: `python3 file_syscall.py`
4. Check that the file exists: `ls`
5. Display its contents: `cat sample.txt`

Program

```
import os
fd = os.open("sample.txt", os.O_CREAT | os.O_WRONLY, 0o644)
data = b"Hello from Linux system calls!"
os.write(fd, data)
os.close(fd)
print("Data written successfully")
```



```
sayan@ubuntu: ~ - nano file_syscall.py
```

```
GNU nano 8.7.1 file_syscall.py *
import os
fd = os.open("sample.txt", os.O_CREAT | os.O_WRONLY, 0o644)
data = b"Hello from linux system calls!"

os.write(fd, data)
os.close(fd)
print("Data written succesfully")
```

^G Help	^O Write Out	^F Where Is	^K Cut
^T Execute	^C Location	M-U Undo	^X Exit
^R Read File	^\\ Replace	^U Paste	^J Justify
^/_ Go To Line	M-E Redo		

Output

sayan@ubuntu: ~

```
sayan@ubuntu:~$ nano file_syscall.py
sayan@ubuntu:~$ python3 file_syscall.py
Data written succesfully
sayan@ubuntu:~$ ls
Documents  Music      Public      Videos      file_syscall.py  process_id.py
Downloads  Pictures   Templates   assignment1  fork_demo.py     process_id.py.save
sayan@ubuntu:~$ cat sample.txt
Hello from linux system calls!sayan@ubuntu:~$
```

Explanation

<i>System call</i>	<i>Purpose</i>
os.open()	Opens the file (creates it if it does not exist) and returns a file descriptor
os.write()	Writes the given bytes to the file
os.close()	Closes the file descriptor

Result: The file sample.txt was created and the text "Hello from Linux system calls!" was written into it using file system calls.

Experiment 5: Reading a File Using System Calls

Aim To read the contents of a file using the system calls open(), read() and close().

Procedure

1. Create the file: nano read_file.py
2. Enter the program given below and save it.
3. Run: python3 read_file.py

Program

```
import os
fd = os.open("sample.txt", os.O_RDONLY)
data = os.read(fd, 100)
os.close(fd)
print("File contents:")
print(data.decode())
```

```
sayan@ubuntu: ~ - nano read_file.py
GNU nano 8.7.1 read_file.py *
import os
fd = os.open("sample.txt", os.O_RDONLY)
data = os.read(fd, 100)

os.close(fd)

print("file contents:")
print(data.decode())

^G Help          ^O Write Out    ^F Where Is    ^K Cut
^T Execute       ^C Location     M-U Undo       ^X Exit
^R Read File     ^\ Replace      ^U Paste       ^J Justify
^/ Go To Line    M-E Redo
```

Output

```
sayan@ubuntu: ~
sayan@ubuntu:~$ nano read_file.py
sayan@ubuntu:~$ python3 read_file.py
file contents:
Hello from linux system calls!
sayan@ubuntu:~$
```

Explanation

The sequence of calls used in this program is:

```
os.open()  
  |  
  v  
os.read()  
  |  
  v  
os.close()
```

Result: The contents of sample.txt were read using open(), read() and close() and displayed on the screen.

Experiment 6: *exec()* System Call

Aim To replace the current process with another program using the `exec()` family of system calls.

Procedure

1. Create the file: `nano exec_demo.py`
2. Enter the program given below and save it.
3. Run: `python3 exec_demo.py`

Program

```
import os

print("Before exec()")

os.execlp("ls", "ls")

print("After exec()")
```

```
sayan@ubuntu: ~ - nano exec_demo.py
```

```
GNU nano 8.7.1
```

```
exec_demo.py *
```

```
import os
print("before exec()")
os.execlp("ls", "ls")
print("after exec()")
```

```
^G Help
^T Execute
^R Read File
^_ Go To Line
```

```
^O Write Out
^C Location
^_ Replace
M-E Redo
```

```
^F Where Is
M-U Undo
^U Paste
```

```
^K Cut
^X Exit
^J Justify
```

Output

```
sayan@ubuntu: ~
```

```
sayan@ubuntu:~$ nano exec_demo.py
```

```
sayan@ubuntu:~$ python3 exec_demo.py
```

```
before exec()
```

```
Documents  Pictures  Videos      file_syscall.py  process_id.py.save  snap
Downloads  Public    assignment1  fork_demo.py     read_file.py        wait_demo.py
Music      Templates exec_demo.py process_id.py    sample.txt
```

```
sayan@ubuntu:~$ █
```

Explanation

After "Before `exec()`" the output of the `ls` command appears. The line `print("After exec()")` is never executed, because `exec()` replaces the current process with the new program.

Python Process

|
exec()
|
v
ls program

The PID stays the same, but the program image running under that PID is replaced.

Result: The Python process was replaced by the ls program using `exec()`, so the statements after `exec()` did not run.

Experiment 7: Complete Process Creation

Aim To combine the process system calls fork(), getpid(), getppid() and wait() in a single program.

Procedure

1. Create the file: nanoproccess_creation.py
2. Enter the program given below and save it.
3. Run: python3 process_creation.py

Program

```
import os

print("Program started")
print("Original PID:", os.getpid())

pid = os.fork()

if pid == 0:
    print("\n--- Child Process ---")
    print("Child PID:", os.getpid())
    print("Parent PID:", os.getppid())

    print("Child is executing...")

else:
    print("\n--- Parent Process ---")
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    os.wait()

    print("Parent: Child has completed")

print("\nProgram completed")
```

```
sayan@ubuntu: ~ -- nano process_creation.py
GNU nano 8.7.1 process_creation.py *

import os
print("program started")
print("original PID:", os.getpid())

pid = os.fork()

if pid == 0:
    print("\n--- child process ---")
    print("child PID:", os.getpid())
    print("parent PID:", os.getppid())
    print("child is executing...")
else:
    print("\n--- parent process ---")
    print("parent PID:", os.getpid())
    print("child PID:", pid)

    os.wait()
    print("parent: child has completed")

print("\nprogram completed")

^G Help          ^O Write Out     ^F Where Is     ^K Cut
^T Execute       ^C Location      M-U Undo        ^X Exit
^R Read File     ^\ Replace       ^U Paste        ^J Justify
^_/ Go To Line   M-E Redo
```

Output

```
sayan@ubuntu: ~
sayan@ubuntu:~$ nano process_creation.py
sayan@ubuntu:~$ python3 process_creation.py
program started
original PID: 6070

--- parent process ---
parent PID: 6070
child PID: 6073

--- child process ---
child PID: 6073
parent PID: 6070
child is executing...

program completed
parent: child has completed

program completed
sayan@ubuntu:~$
```

Explanation

The parent prints its own PID and the child PID, then waits for the child. The child prints its own PID and its parent PID.

Result: A child process was created with `fork()`, the parent waited for it with `wait()`, and the process IDs were displayed using `getpid()` and `getppid()`.

Important Linux Commands for the Lab

<i>Command</i>	<i>Purpose</i>
pwd	Show current directory
ls	List files
cd	Change directory
mkdir	Create directory
touch	Create file
cat	Display file
nano	Edit file
cp	Copy file
mv	Move/rename file
rm	Delete file
ps	Display processes
clear	Clear terminal
python3 file.py	Execute Python program

Conclusion

In these experiments, the process system calls `getpid()`, `getppid()`, `fork()`, `wait()` and `exec()`, and the file system calls `open()`, `write()`, `read()` and `close()`, were run through Python's `os` module on Linux. The experiments show how a process is created, how a parent synchronizes with its child, how a process image is replaced, and how files are handled at the system call level.